Atomic floating-point min/max

Document number: P3008R3.
Date: 2024-11-18.
Authors: Gonzalo Brito Gadeschi, David Sankel <dsankel@adobe.com (mailto:dsankel@adobe.com)>.
Reply to: gonzalob *at* nvidia.com (http://nvidia.com) .
Audience: LEWG.

Table of Contents

Changelog

- **Revision 2** (post SG6 review): SG6 took the following poll and the paper is updated accordingly:

  - Poll 1: Accept SG1 proposal as listed in P3008R1 with "implementation-defined" instead of "unspecified" and forward to LEWG.

SF	F	N	A	SA
2	3	2	2	0

    Consensus.

        - A: preference for no fetch_min/max default, but rather explicit fetch_fminimum/minimum_num
        - A: implementation-define may cause pessimistic implementations.
        - SF: two authors.

    - Poll 2: If one of the input is a NaN then it is "unspecified" whether the result value is either a NaN or the other value.

SF	F	N	A	SA
2	2	3	0	1

      Consensus.
      SA: bad for teachability.

    - Poll 3: With respect to signaling NaNs, SG6 recommends:

        - for `fetch_fminimum` / `fminimum_num` / `fmaximum` / `fmaximum_num` to be specified in terms of their corresponding C23 APIs (ignore signaling NaNs in the wording, refer to C23), and
        - for fetch_min/max, to not explicitly specify signaling NaN behavior (ignore signaling NaNs in the wording)

SF	F	N	A	SA
4	3	2	0	0

      Consensus.

- **Revision 1** (post SG1 review):
    - Update to account for removal of min/max from P0493.
    - Update C17 to C23 wording for signaling NaNs.
    - SG1 DIRECTION POLL:

        1. The default fetch_min(T, mo=SC)->T, fetch_max(T, mo=SC)->T should exist

        2. The default fetch_min, fetch_max should have these semantics:
           * Which value is stored in the atomic is unspecified if an input is a NaN
           * Implementations should treat -0 as less than +0 (use normative encouragement)
           * (Note: must *not* require std::min/max)

3. Users need a way to specify semantics (up to LEWG how to achieve this):
   * Must-have: fminimum/fmaximum
   * Must-have: fminimum_num/fmaximum_num
   * Can have if LEWG desires it but we do not: std::min/max

SF	F	N	A	SA
5	6	2	0	0

Unanimous consent

- **Revision 0**: initial revision.

Introduction

P0493R4 (https://wg21.link/P0493R4) *Atomic minimum/maximum* proposes the addition of `atomic<T>::fetch_min` and `atomic<T>::fetch_max` operations, which atomically perform `std::min` and `std::max` computations.

The Varna '23 plenary rejected these semantics for `atomic<floating-point>` types due to safety concerns. These semantics deviate from the current standard practice, as defined by IEEE 754-2019 (https://ieeexplore.ieee.org/document/8766229) recommendations and the capabilities of available hardware.

These operations were removed from subsequent revisions of P0493 (https://wg21.link/P0493). This paper proposes adding these operations to C++ with different semantics. It reviews the semantics proposed in P0493R4 (https://wg21.link/P0493R4) and standard practice: IEEE 754-2008 (https://iremi.univ-reunion.fr/IMG/pdf/ieee-754-2008.pdf), IEEE 754-2019 (https://ieeexplore.ieee.org/document/8766229), and available hardware support. It then explores the design space and evaluates alternative solutions.

Finally, the authors propose two coherent alternatives and concrete wording changes implementing some of the semantics IEEE 754-2019 (https://ieeexplore.ieee.org/document/8766229) recommends all vendors implement.

Survey of programming language floating-point min/max API semantics

This section provides an overview of widely used floating-point `min` / `max` semantics. We will focus on how various `min` / `max` semantics handle the following "corner cases":

- **Signed zero**: Should `-0` be considered less than `+0` (`-0 < +0`), or should `-0` and `+0` be treated as equivalent (`-0 == +0`) such that, e.g., `min(+0, -0) = +0` ?
- **One quiet NaN** (qNaN): when one of the arguments is a qNaN, should it be treated as *Missing Data* and the other argument returned, or should it be treated as an *Error* and propagated?
  - **Missing Data**: returns the other argument: `min(x, qNaN) = x` and `min(qNaN, x) = x` .
  - **Error**: propagates the qNaN: `min(x, qNaN) = qNaN` and `min(qNaN, x) = qNaN` .

Explicitly and intentionally, this paper ignores:

- **Signaling NaNs**: C23 specifies that their full support is not required, and where their specification is not provided leaving them as implementation-defined behavior (either as quiet NaN or signaling NaN). C++ does not specify anything about them beyond what C23 covers. An analysis of signaling NaNs and their impact on this proposal is left to a future version of this paper and captured as a current unresolved question in a latter section of this proposal.
- **NaN payload propagation**: IEEE 754 does not mandate it, and programming languages do not interpret NaN payloads.

Table 1 summarizes the semantics of the different C or C++ APIs regarding Signed Zeros and Quiet NaN handling and documents whether implementations that implement particular IEEE 754 semantics are valid implementations of the APIs.

C API	C++ API	Signed Zeros	One qNaN	IEEE 754 impl valid?
-	`min` `max`	Equivalent	Undefined [1]	Ternary [0]

`fmin`
`fmax`
	-	Equivalent or `-0 < +0` (QoI) [2]	Missing Data	`minNum`
				`maxNum`
				or (QoI)
				`minimumNumber`
				`maximumNumber`

`fminimum` `fmaximum`	-	`-0 < +0`	Error	`minimum` `maximum`
`fminimum_num` `fmaximum_num`	-	`-0 < +0`	Missing Data	`minimumNumber` `maximumNumber`

[0] Ternary semantics: `min(x,y) = y < x? y : x`, `max(x,y) = x < y? y : x`; both return the first argument if the arguments are equivalent or one argument is a NaN.
[1] In practice, the same implementation as for valid values is used, and the first argument passed to the function is returned (the second argument of the ternary expression is picked, since the conditional involving a NaN is always false).
[2] QoI: "Quality of Implementation", i.e., `fmin` does not require `-0 < +0`, but it recommends that high quality implementations implement it.

C++ `std::min` / `std::max`

The `std::min` and `std::max` algorithms have a *precondition* on their arguments ([alg.min.max.1] (http://eel.is/c++draft/alg.min.max#1)):

> Preconditions: For the first form, `T` meets the Cpp17LessThanComparable (http://eel.is/c++draft/utility.arg.requirements#tab:cpp17.lessthancomparable) requirements (Table Cpp17LessThanComparable (http://eel.is/c++draft/utility.arg.requirements#tab:cpp17.lessthancomparable)).

which NaNs do not satisfy.

▶ Rationale.

For valid values, implementations follow ([alg.min.max] (http://eel.is/c++draft/alg.min.max)):

> [`std::min`]:
> *Returns*: The smaller value. Returns the first argument when the arguments are equivalent.

> [`std::max`]:
> *Returns*: The larger value. Returns the first argument when the arguments are equivalent.

which preserves equivalence between `-0` and `+0` (godbolt (https://cpp.godbolt.org/z/84j4Kse46)):

```
// Listing 1: std::min/std::max
auto min = [](auto& a, auto& b) -> auto& { return std::min(x, y); }; // y <
auto max = [](auto& a, auto& b) -> auto& { return std::max(x, y); }; // x <

min(qNaN, 2.f); // UB: qNaN
max(qNaN, 2.f); // UB: qNaN
min(2.f, qNaN); // UB: 2
max(2.f, qNaN); // UB: 2
min(−0.f, +0.f); // −0
max(−0.f, +0.f); // −0
min(+0.f, −0.f); // +0
max(+0.f, −0.f); // +0
```

In all standard library implementations surveyed, the manifestation of the *undefined behavior* is to return the first argument; this is depicted in the example with "UB: qNaN" and "UB: 2".

The behavior of these semantics are:

- **Signed Zero**: undefined since not totally ordered.
- **One qNaN**: undefined since not totally ordered.

In concurrent programs, these semantics become even less intuitive, e.g., the sign of the result of a concurrent computation may depend on the order in which an observer sitting on top of the memory location observes the operations from different threads. If negative and positive zero are equivalent, the sign of this final result may differ across executions.

C `fmin` / `fmax`

The semantics of the ISO/IEC 9899:2024 (C23) standard (https://open-std.org/JTC1/SC22/WG14/www/docs/n3096.pdf) `fmin` / `fmax` functions are compatible with implementations of IEEE 754-2008 (https://iremi.univ-reunion.fr/IMG/pdf/ieee-754-2008.pdf) `minNum` and `maxNum` and of IEEE 754-2019 (https://ieeexplore.ieee.org/document/8766229) `minimumNumber` / `maximumNumber`. The `fmin` / `fmax` functions are available in C++ through the `<cmath>` header as `std::fmin/std::fmax`.

The semantics of C's `fmin` / `fmax`

- **Signed zero**:
  - `−0` may be equivalent to `+0` (`minNum` / `maxNum`)
  - `−0 < +0` allowed as QoI (`minimumNumber` / `maximumNumber` guarantee it).
- **One qNaN**: missing data (other value returned).

▶ (collapsible) C23 fmin/fmax specification.

▶ (collapsible) [IEEE 754-2008] minNum/maxNum specification.

That is, in the presence of a qNaN, these return the value, but signed zeros may or may not be equivalent:

```
// Listing 2: fmin/fmax
fmin(qNaN, 2.f): 2
fmax(qNaN, 2.f): 2
fmin(2.f, qNaN): 2
fmax(2.f, qNaN): 2
fmin(-0.f, +0.f): -0 or +0
fmax(-0.f, +0.f): -0 or +0
fmin(+0.f, -0.f): -0 or +0
fmax(+0.f, -0.f): -0 or +0
```

IEEE 754-2019 (https://ieeexplore.ieee.org/document/8766229) removed `minNum` and `maxNum` operations and replaced them with `minimumNumber` / `maximumNumber` and `minimum` / `maximum`. These are surveyed in the next section. The gist of the rationale for their removal from The Removal/Demotion of `MinNum` and `MaxNum` Operations from IEEE 754-2018 (https://grouper.ieee.org/groups/msc/ANSI_IEEE-Std-754-2019/background/minNum_maxNum_Removal_Demotion_v3.pdf) is:

> These [minNum, maxNum] operations are removed from or demoted in IEEE std 754-2018, due to their non-associativity. [...] With this non-associativity, different compilations or runs on parallel processing can return different answers [...].

C23 `minimum` / `maximum` / `minimumNumber` / `maximumNumber`

IEEE 754-2019 (https://ieeexplore.ieee.org/document/8766229) removed `minNum` / `maxNum` and recommends programming languages to provide their replacements instead: `minimumNumber` / `maximumNumber` / `minimum` / `maximum`.

▶ (collapsible) IEEE 754-2019 minimum/maximum/minimumNumber/maximumNumber specification.

▶ (collapsible) C23 fmaximum/fminimum/fmaximum_num/fminimum_num specification.

The semantics of these functions matches for signed zero: `-0 < +0`, and differs in their treatment when one argument is a qNaN:

- **Missing Data**: `fminimumNumber` / `fmaximumNumber`, return the Number.

- **Errors**: `fminimum` / `fmaximum` , propagate the qNaN.

```
// Listing 3: fminimum/fmaximum
fminimum(qNaN, 2.f): qNaN
fmaximum(qNaN, 2.f): qNaN
fminimum(2.f, qNaN): qNaN
fmaximum(2.f, qNaN): qNaN
fminimum(-0.f, +0.f): -0
fmaximum(-0.f, +0.f): +0
fminimum(+0.f, -0.f): -0
fmaximum(+0.f, -0.f): +0

fminimum_num(qNaN, 2.f): 2
fmaximum_num(qNaN, 2.f): 2
fminimum_num(2.f, qNaN): 2
fmaximum_num(2.f, qNaN): 2
fminimum_num(-0.f, +0.f): -0
fmaximum_num(-0.f, +0.f): +0
fminimum_num(+0.f, -0.f): -0
fmaximum_num(+0.f, -0.f): +0
```

Impact of replacing `min` with `fminimum_num`

Table 2 captures the impact of replacing `min` with `fminimum_num` is as follows:

std::min / std::max	**fminimum_num / fmaximum_num**

`min(qNaN, 2.f); // UB: qNaN`
`max(qNaN, 2.f); // UB: qNaN`
`min(+0.f, -0.f); // +0`
`max(-0.f, +0.f); // -0`
	`fminimum_num(qNaN, 2.f); // 2`
	`fmaximum_num(qNaN, 2.f); // 2`
	`fminimum_num(+0.f, -0.f); // -0`
	`fmaximum_num(-0.f, +0.f); // +0`

That is, when the first input of `std::min` / `std::max` is a qNaN, then these switch from exhibiting *undefined behavior* to returning a number, and when signed zeros are involved, there is a case where the result has a different sign.

Survey of hardware atomic floating-point min/max API semantics

On systems without native support for atomic floating-point min/max operations, these operations must be performed atomically, e.g., using a CAS loop, a compare-and-conditional-store loop, an LL/SC loop, or, e.g., by taking a lock, which would make the atomic not lock-

free. In these systems, the memory latency overhead may outweight the cost of performing the actual *arithmetic* portion of the min/max operations, and extra effort may be required to ensure forward progress properties like starvation freedom, and therefore those systems are not considered here.

Table 3 surveys the publicly known Instruction Set Architectures (ISAs) with atomic floating-point min/max operations, which are all GPU ISAs:

Vendor	**ISA**	**Instructions**
AMD	CDNA2 (https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/instruction-set-architectures/instinct-mi200-cdna2-instruction-set-architecture.pdf)+	MIN (https://www.am -tech-docs/instr instruction-set- MAX (https://www.am -tech-docs/instr instruction-set-
Intel	Xe ISA (https://www.intel.com/content/www/us/en/docs/graphics-for-linux/developer-reference/1-0/alchemist-arctic-sound-m.html)	`AOP_FMIN` `AOP_FMAX` [0
NVIDIA	PTX (https://docs.nvidia.com/cuda/parallel-thread-execution/index.html)	atom execution/index. communication- red execution/index. communication-
Neutral [1]	SPIR-V (http://htmlpreview.github.io/?https://github.com/KhronosGroup/SPIRV-Registry/blob/main/extensions/EXT/SPV_EXT_shader_atomic_float_min_max.html) Extension	OpAtomicFM V/specs/unified1 OpAtomicFM V/specs/unified1

- [0]: Volume 2d: Command Reference: Structures, page 229.
- [1]: The Vulkan extension corresponding to this SPIR-V extension is `VK_EXT_shader_atomic_float2`. Hardware that implements it is listed here (https://vulkan.gpuinfo.org/listdevicescoverage.php?extensionname=VK_EXT_shader_atomic_float2&extensionfeature=shaderBufferFloat32AtomicMinMax&platform=all) and here (https://vulkan.gpuinfo.org/listdevicescoverage.php?extensionname=VK_EXT_shader_atomic_float2&extensionfeature=shaderSharedFloat32AtomicMinMax&platform=all).

All the architectures surveyed order `-0 < +0`, treat qNaNs as missing data, and implement IEEE 754-2019 `minimumNumber` and `maximumNumber` semantics. The SPIR-V extension requires C `fmin` / `fmax` semantics, which allows `-0 < +0` but does not require it.

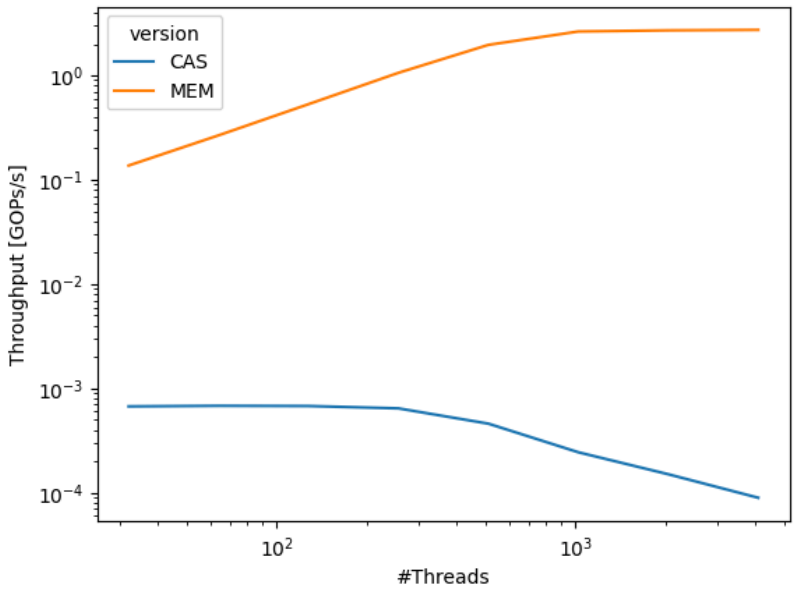
The semantics vendor implement are compatible with C23's `fminimum_num` / `fmaximum_num`, and C's `fmin` / `fmax`, but not with C++'s `std::min` / `std::max` due to the different outcomes when signed-zeros are involved.

Performance impact of `atomic<floating-point>::fetch_min` / `_max` semantics

We compare the performance impact of two different `atomic<floating-point>::fetch_max` semantics:

- `std::min`, which lowers to a CAS-loop that performs `std::min` (similar for compare-and-conditional-store), and
- `fminimum_num`, which lowers to native atomic operations,

using a synthetic micro-benchmark that measures throughput in Giga Operations per Second (y-axis; logarithmic) as a function of the number of threads (x-axis; logarithmic) from 32 to 130'000 hardware threads on an NVIDIA GPU system.



Modern concurrent systems provide dozens of millions of hardware threads operating on the same shared memory.

While native in-memory atomic operations increase throughput until its theoretical peak with just a few thousand threads, the performance of compare and swap strategies decreases as the number of threads increases due to excess contention. At just a few thousand threads,

the performance is already four orders of magnitude worse (~10'000x) than that of native in-memory operations. This is why vendors of highly concurrent systems provide these operations.

While most CPU ISAs lack native hardware instructions for atomic floating-point min/max, their performance impact is expected to be similar to that of atomic integer min/max. Section 9 of P0493R4 (https://wg21.link/P0493R4) presents benchmarks comparing the performance of a CAS-based implementation against an Arm v8.1 implementation using the native atomic `ldsmaxl` instruction. The benchmark covers a range of cores, from 2 to 64. The performance improvement of the native instruction ranges between 2.75x (2 cores) to 1.7x at 64 cores. We expect this behavior to transfer to atomic floating-point min/max.

We, unfortunately, do not have benchmarks for other hardware configurations at this time.

Design space

SG1 (Concurrency) and SG6 (Numerics) agreed on the following semantics (see polls in the Changelog of Revision's 1 and 2 of this paper):

1. The "default" `fetch_min(T, mo=SC)->T` and `fetch_max(T, mo=SC)->T` APIs should exist and have these semantics:
   - Which value is stored in the atomic is *unspecified* if an input is a NaN.
   - Implementations *should* treat `-0` as less than `+0` (normative encouragement; must *not* require std::min/max).
   - No explicit specification of signaling NaN behavior.
2. Users need a way to specify `fminimum` / `fmaximum` and `fminimum_num` / `fmaximum_num` semantics. These APIs should be specified in term of the corresponding C23 APIs.
3. Can have APIs with `std::min` / `std::max` semantics that leave the handling of signed zero and NaNs as undefined, but SG1 and SG6 do not want these APIs.

These semantics are different from `std::min` / `std::max` semantics, which leave the handling of signed-zero and NaNs as undefined behavior, due to these values not being totally ordered. There is precendent in the C++ standard for the atomic operations semantics to deviate from the non-atomic semantics. For example, `atomic<int>::fetch_add` wraps around on overflow, instead of exhibiting undefined behavior, see [atomics#ref.int-6] (http://eel.is/c++draft/atomics#ref.int-6):

> *Remarks*: For signed integer types, the result is as if the object value and parameters were converted to their corresponding unsigned types, the computation performed on those types, and the result converted back to the signed type.
>
> [Note 2: There are no undefined results arising from the computation. — end note]

The wording section proposes the actual API.

Wording

Add the following to [atomics.ref.float] (https://eel.is/c++draft/atomics.ref.float) immediately below `fetch_sub`:

```
namespace std {
template <> struct atomic_ref<floating-point> {
  floating-point fetch_max(floating-point, memory_order = memory_order::seq_cst) c
  floating-point fetch_min(floating-point, memory_order = memory_order::seq_cst) c
  floating-point fetch_fmaximum(floating-point, memory_order = memory_order::seq_c
  floating-point fetch_fmininum(floating-point, memory_order = memory_order::seq_c
  floating-point fetch_fmaximum_num(floating-point, memory_order = memory_order::s
  floating-point fetch_fmininum_num(floating-point, memory_order = memory_order::s
};
}
```

```
floating-point-type fetch_key(floating-point-type operand,
                          memory_order order = memory_order::seq_cst) const noexce
```

- Effects: Atomically replaces the value referenced by `*ptr` with the result of the computation applied to the value referenced by `*ptr` and the given operand. Memory is affected according to the value of order. These operations are atomic read-modify-write operations ([intro.races]).
- Returns: Atomically, the value referenced by *ptr immediately before the effects.
- Remarks: If the result is not a representable value for its type ([expr.pre]), the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on floating-point-type should conform to the `std::numeric_limits<floating-point-type>` traits associated with the floating-point type ([limits.syn]). The floating-point environment ([cfenv]) for atomic arithmetic operations on floating-point-type may be different than the calling thread's floating-point environment.
- Remarks:

    - For `fetch_fmaximum` and `fetch_fminimum`, the maximum and minimum computation is performed as if by `std::fmaximum` and `std::fminimum`, with the object value and the first parameter as the arguments.
    - For `fetch_fmaximum_num` and `fetch_fminimum_num`, the maximum and minimum computation is performed as if by `std::fmaximum_num` and `std::fminimum_num`, with the object value and the first parameter as the arguments.
    - For `fetch_max` and `fetch_min`, the maximum and minimum computation is performed as if by `std::fmaximum_num` and `std::fminimum_num`, with the object value and the first parameter as the arguments. If the given operand or the value referenced by `*ptr` are NaN, which of these values is stored at `*ptr` is unspecified. If the given operand and the value referenced by `*ptr` are differently signed zeros, which of these values is stored at `*ptr` is unspecified.
- Recommended practice: The implementation of `fetch_max` and `fetch_min` should treat negative zero as smaller than positive zero.

Add the following to [atomics.types.float] (https://eel.is/c++draft/atomics.types.float) immediately below `fetch_sub`:

```
namespace std {
template <> struct atomic<floating-point> {
  floating-point fetch_max(floating-point, memory_order = memory_order::seq_cst) v
  floating-point fetch_max(floating-point, memory_order = memory_order::seq_cst) r
  floating-point fetch_min(floating-point, memory_order = memory_order::seq_cst) v
  floating-point fetch_min(floating-point, memory_order = memory_order::seq_cst) r
  floating-point fetch_fmaximum(floating-point, memory_order = memory_order::seq_c
  floating-point fetch_fmaximum(floating-point, memory_order = memory_order::seq_c
  floating-point fetch_fmininum(floating-point, memory_order = memory_order::seq_c
  floating-point fetch_fmininum(floating-point, memory_order = memory_order::seq_c
  floating-point fetch_fmaximum_num(floating-point, memory_order = memory_order::s
  floating-point fetch_fmaximum_num(floating-point, memory_order = memory_order::s
  floating-point fetch_fmininum_num(floating-point, memory_order = memory_order::s
  floating-point fetch_fmininum_num(floating-point, memory_order = memory_order::s
};
}
```

```
T fetch_key(T operand, memory_order order = memory_order::seq_cst) volatile noexce
T fetch_key(T operand, memory_order order = memory_order::seq_cst) noexcept;
```

- Constraints: For the volatile overload of this function, is_always_lock_free is true.

- Effects: Atomically replaces the value pointed to by this with the result of the computation applied to the value pointed to by this and the given operand. Memory is affected according to the value of order. These operations are atomic read-modify-write operations ([intro.multithread]).
- Returns: Atomically, the value pointed to by this immediately before the effects.
- Remarks: If the result is not a representable value for its type ([expr.pre]) the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on floating-point-type should conform to the std::numeric_limits<floating-point-type> traits associated with the floating-point type ([limits.syn]). The floating-point environment ([cfenv]) for atomic arithmetic operations on floating-point-type may be different than the calling thread's floating-point environment.
- Remarks:
  - For `fetch_fmaximum` and `fetch_fmininimum`, the maximum and minimum computation is performed as if by `std::fmaximum` and `std::fminimum`, with the object value and the first parameter as the arguments.
  - For `fetch_fmaximum_num` and `fetch_fmininimum_num`, the maximum and minimum computation is performed as if by `std::fmaximum_num` and `std::fminimum_num`, with the object value and the first parameter as the arguments.
  - For `fetch_max` and `fetch_min`, the maximum and minimum computation is performed as if by `std::fmaximum_num` and `std::fminimum_num`, with the object value and the first parameter as the arguments. If the given operand or the object value are NaN, which of these values replaces the object value is unspecified. If the given operand and the object value are differently signed zeros, which of these values replaces the object value is unspecified.
- Recommended practice: The implementation of `fetch_max` and `fetch_min` should treat negative zero as smaller than positive zero.

Update `__cpp_lib_atomic_min_max` version macro in `<version>` synopsis [version.syn] (https://eel.is/c++draft/version.syn#2) to the C++ version this feature is introduced in:

```
#define __cpp_lib_atomic_min_max ~~202403L~~______L // freestanding, also in <atomic>
```

EDITORIAL NOTES:

- This paper relies on the C23 math functions which have not been merged into C++26 draft yet (see P3384 (https://wg21.link/p3384)).

References

1. P0493R4 Atomic maximum/minimum (https://wg21.link/P0493R4).
2. The Removal/Demotion of MinNum and MaxNum Operations from IEEE 754™-2018 (https://grouper.ieee.org/groups/msc/ANSI_IEEE-Std-754-2019/background/minNum_maxNum_Removal_Demotion_v3.pdf).
3. Min-max functions (https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2273.pdf) Proposal for TS 18661 update WG14 N2273.
4. ISO/IEC 9899:2024 C23 standard (https://open-std.org/JTC1/SC22/WG14/www/docs/n3096.pdf).
5. ANSI/IEEE Std 754-2008 (https://iremi.univ-reunion.fr/IMG/pdf/ieee-754-2008.pdf).
6. ANSI/IEEE Std 754-2019 (https://ieeexplore.ieee.org/document/8766229).